A Location Based Smart Drive

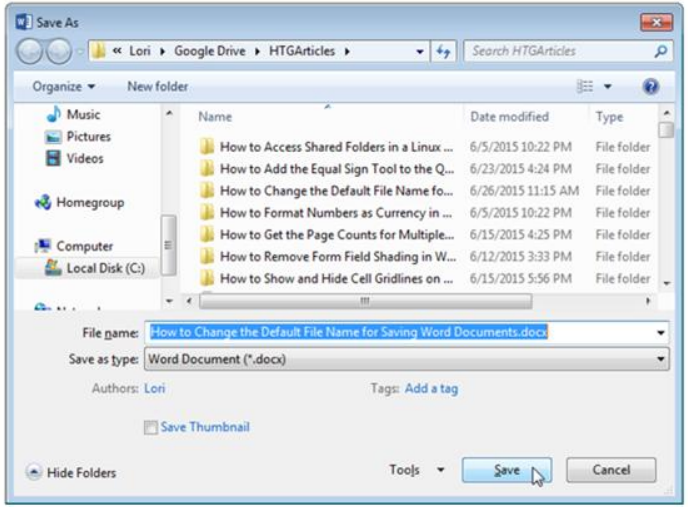
1) Taimoor Shahzad - tshahzad@uvic.ca

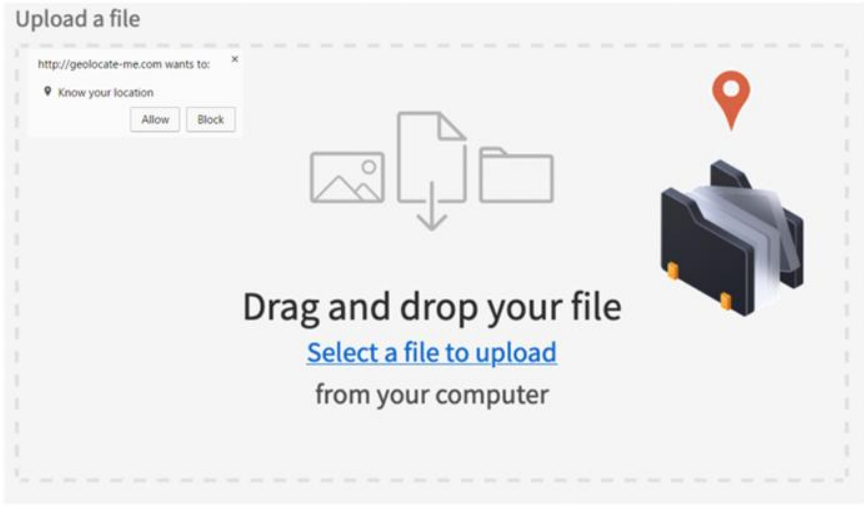
2) Lucy Blecha - lucyblecha@uvic.ca

3) Suhan Yoon - syoon001@uvic.ca

4) Guangning Li - guangningli@uvic.ca

Typical Drive vs Location Based Smart Drive

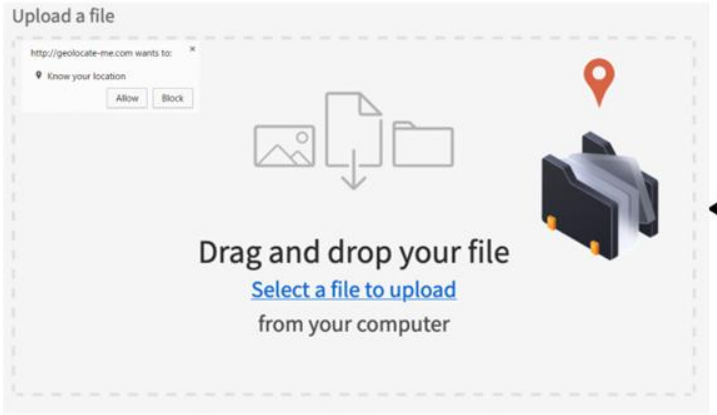




- **Organization**: Users manually create folders and sort files
- **File Naming**: Users assign file names and folder labels manually
- **Geolocation Integration**: No location based file sorting is present
- **User Input**: High reliance on user effort for maintaining structured file organization
- **Use Case**: General purpose storage without specific customization or contextual organization

- **Location Based**: Automatically organizes uploaded files into folders named after the city where the upload occurs, using geolocation data
- **Integration with Geolocation APIs**: Utilizes tools like the OpenCage Reverse Geocoding API to convert latitude and longitude into city names
- **Fallback for Manual Input**: Provides a robust fallback mechanism for users to manually input city names
- **Time Saving Automation:** Minimizes user effort by automating file sorting

Technology Analysis











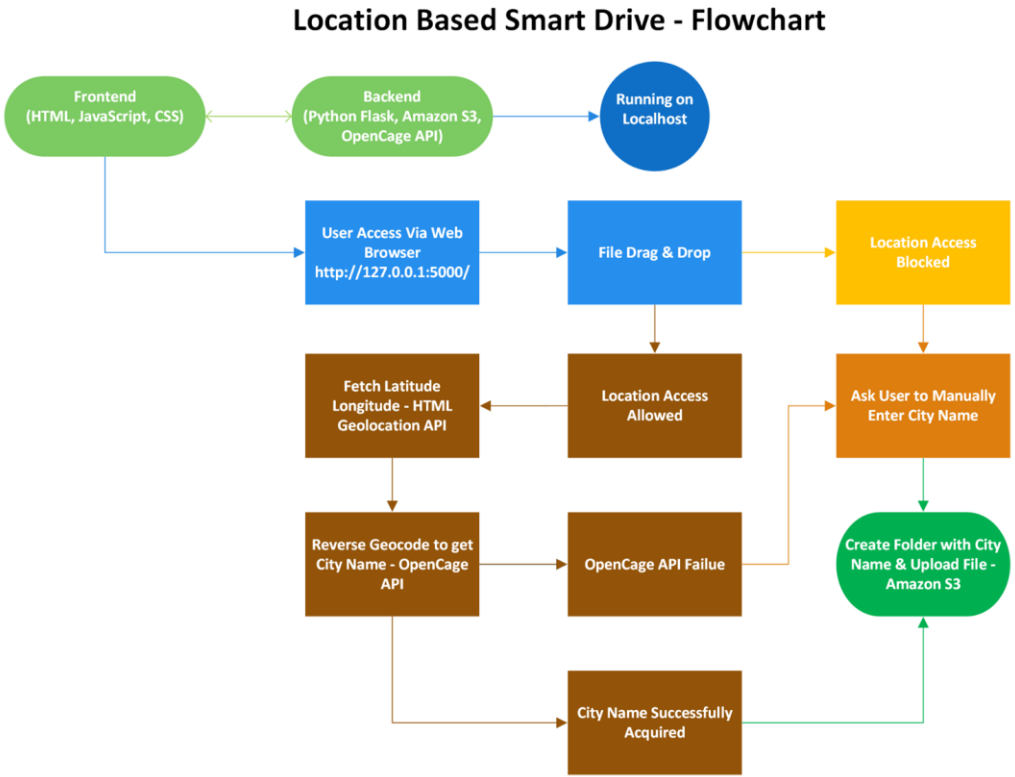
System Design & Architecture

- Smart Drive System is designed to automate file storage and organization by using real-time location data to sort files into folders named after the city where they are uploaded.
- Capable of handling both automatic and manual city name inputs for folder creation.
- Fallback Mechanism Integrated to handle API failures/blocked location access.
- Frontend (HTML, JavaScript, CSS)
- Backend (Python Flask, Amazon S3 Cloud, Opencage API)

Data Flow

- Server is started on localhost and accessed by user on any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge) using address: http://127.0.0.1:5000/
- User uploads file and location access is either granted or blocked.
- If location access is allowed, city name is fetched using Geolocation API and OpenCage API and folder with city name is created and file is uploaded on backend on Amazon S3 bucket.
- If location access is blocked or if API is fails to fetch the city name, user is prompted to enter the city name manually and folder with manual city name is created and file is uploaded on backend on Amazon S3 bucket.

System Flowchart



System Implementation

- Frontend developed (HTML, JavaScript, CSS) with a user-friendly drag/drop interface.
- Frontend integrated with the HTML Geolocation API to fetch longitude and latitude data.
- User uploads file and location access prompt appears. Reverse geocoding is carried out by OpenCage API to convert the positional data into a city name.
- In case of blockage of location access or failure of API to fetch the city name, user can enter the city name manually.
- Folder is created with city name and file is uploaded on backend on Amazon S3 bucket.

Languages/Libraries/APIs/Cloud

- HTML, Javascript, CSS, Python Flask
- HTML Geolocation API
- **boto3** for Amazon S3 integration..
- **requests** for OpenCage API calls
- Amazon S3 Cloud Storage

Testing

Test Case 1: Real time location : (Vancouver and Victoria) Pass

Test Case 2: Automatic folder creation : Pass

Test Case 3: Any popular web browser : (Chrome, Firefox, Edge) Pass

Test Case 4: Multiple Device : (Smartphone and PC) Pass

Test Case 5: Failure to find location of file: Pass

Test Case 6: Different File types: (PNG, TXT, PDF) Pass

Data safety and security considerations

- Data encryption
- User Authentication and Access Control
- Data Backup and Recovery with AWS
- Preventing Data Leakage
- System Updates and Vulnerability Management



Data encryption & Authentication and Access Control

- Data transmission encryption: TLS or HTTPS
- Data storage encryption: AES-256, Amazon S3 built-in server-side encryption (SSE)
- Decryption of files occurs only after successful user authentication.
- Multi-Factor Authentication (MFA)
    - Combine a password with a dynamic verification code
    - Use AWS – IAM MFA feature
- Role-Based Access Control
    - Use AWS IAM to define permissions for different user groups
    - Restrict access to sensitive actions



Data Backup and Recovery & Preventing Data Leakage & System Updates

- Real-time Backup
  - Amazon S3 and Amazon EBS
  - AWS Backup

- Disaster Recovery
  - Hot backup: S3 Cross-Region Replication, DynamoDB
  - Cold backup: Amazon S3 Glacier



- Data Loss Prevention (DLP) Monitoring
  - Monitor the system in real-time
  - Use AWS Macie to identify and classify sensitive data in Amazon S3 buckets
  - AWS CloudTrail and Amazon GuardDuty monitor account activities and detect unusual behaviors

- Regular Software Updates

- Penetration Testing

Timelines, resource allocation and project planning

- Project timeline
- Resource Allocation
- Project Planning



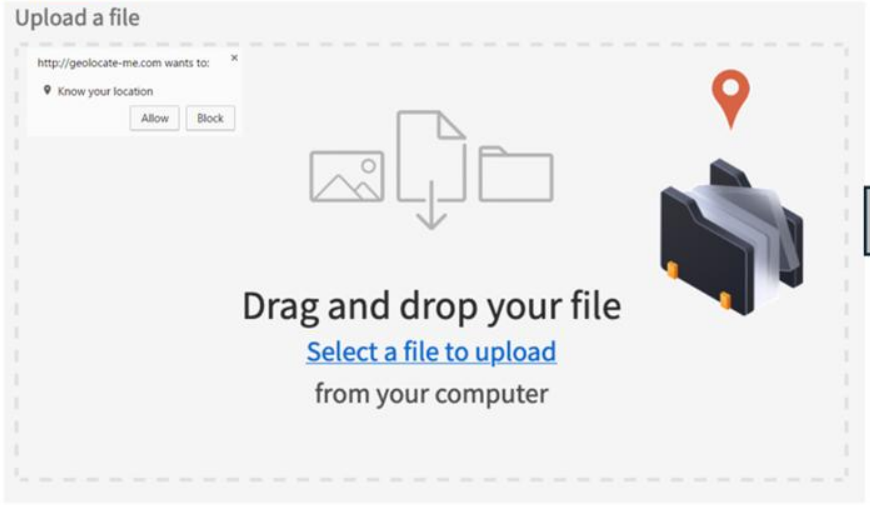
Resource Allocation & Timeline

Task	Assigned Member	Description	Timeline
Project Background, Technology & Market Analysis	Suhan Yoon	Conducted research on the project's scope, relevant technologies, and potential market demand.	Nov. 1 - Nov. 5
System Design and Architecture	Taimoor Shahzad	Designed system components and interactions, focusing on scalability and cloud-based solutions.	Nov. 6 - Nov. 10
System Implementation Approach	Taimoor Shahzad	Developed and implemented the portal using AWS services, drag-and-drop functionality, and automation.	Nov. 11 - Nov. 17
Data Safety and Security Considerations	Guangning Li	Ensured encryption, access control, and data loss prevention measures were in place.	Nov. 18 - Nov. 24
Testing Plan, Test Cases, and Results	Lucy Blecha	Designed and executed comprehensive tests, including functional, security, and stress testing.	Nov. 24 - Nov. 29

Summary/Conclusion



Idea/Brainstorming



Outcome/Result



